

RPAL Interpreter

Project Work Plan & Task Allocation

Group Project | 2 Members

1. Project Overview

This project requires building a complete RPAL language interpreter from scratch using Java (or C/C++). It consists of four sequential phases: a Lexical Analyzer, a Parser that produces an Abstract Syntax Tree (AST), an AST-to-Standardized Tree (ST) converter, and a CSE Machine evaluator. No parser-generation tools (lex, yacc, etc.) may be used.

Components at a Glance

Component	Input	Output	Grading Weight
Lexical Analyzer	Raw RPAL source file	Token stream	~15%
Parser	Token stream	Abstract Syntax Tree (AST)	~20%
AST → ST	AST	Standardized Tree (ST)	~15%
CSE Machine	ST + Control structures	Evaluated output	~20%
Report & Comments	—	PDF report + code comments	30%

2. Task Allocation by Member

The project is split evenly. Member 1 handles the front-end (Lexer + Parser → AST) and shared infrastructure. Member 2 handles the back-end (AST → ST + CSE Machine) and the report. Both members collaborate on testing and integration.

Member 1 – Lexer + Parser (Front-end)

#	Task	Details	Est. Hours	Deadline
M1-01	Project Setup & Structure	Create repo, folder layout (lexer/, parser/, ast/, cse/), Makefile/build scripts, base Node/class skeleton	3 hrs	Day 1
M1-02	Lexical Analyzer – Tokenizer	Implement Lexer class: read source char-by-char, produce tokens (ID,	6 hrs	Day 3

		INT, STRING, KEYWORD, OP, PUNCT, EOF). Handle whitespace & // comments		
M1-03	Token Type Definitions	Define all RPAL keywords (let, in, fn, where, aug, rec, and, or, not, eq, ne, ls, gr ...) and operator symbols	2 hrs	Day 3
M1-04	Parser – Recursive Descent	Implement one method per grammar rule (E, Ew, T, Ta, Tc, B, Bt, Bs, Bp, Bf, A, At, Af, Ap, R, Rn, D, Da, Dr, Db, Vb, Vl). Build ASTNode tree	12 hrs	Day 7
M1-05	AST Node Structure	Define ASTNode (type, value, List<ASTNode> children). Implement tree printer matching rpal.exe -ast output format	3 hrs	Day 6
M1-06	Lexer + Parser Unit Tests	Write test cases for each token type; test parser against provided sample programs; verify AST output matches rpal.exe	4 hrs	Day 8
M1-07	Integration & Main Entry Point	Wire Lexer → Parser → ST converter → CSE in main(). Handle command-line arg (file_name). Graceful error messages	3 hrs	Day 13

Member 2 – ST Converter + CSE Machine (Back-end)

#	Task	Details	Est. Hours	Deadline
M2-01	AST → Standardized Tree (ST)	Implement standardization rules for: let, where, lambda, tau, rec, within, and, @. Traverse AST recursively and rewrite nodes	8 hrs	Day 10
M2-02	ST Verification	Compare ST output with rpal.exe -st flag on all sample	3 hrs	Day 10

		programs. Fix mismatches		
M2-03	CSE Machine – Control Structures	Flatten ST into control structures (delta arrays). Build environment model (env chain). Define stack-machine state	6 hrs	Day 12
M2-04	CSE Machine – Evaluation Rules	Implement all CSE rules: name lookup, lambda apply, conditional, tuple, built-in functions (Print, Order, Stern, Stem, Conc, ItoS, etc.)	10 hrs	Day 13
M2-05	Built-in Functions & I/O	Implement RPAL built-ins: Print, Isinteger, Isstring, Istruthvalue, Istuple, Isfunction, Isdummy, Order, Null, Stem, Stern, Conc, ItoS	4 hrs	Day 13
M2-06	End-to-End Testing	Run all test programs; diff output vs rpal.exe expected output. Fix failures. Test edge cases (recursion, nested lambdas, tuples)	5 hrs	Day 14
M2-07	Report (PDF)	Write PDF report: student names & IDs, program structure, function prototypes, design decisions, known issues	4 hrs	Day 15

3. Shared Responsibilities

- **Code Review:** Each member reviews the other's code before integration.
- **Integration Testing:** Run all provided RPAL test programs; compare output with rpal.exe using diff.
- **Makefile:** Ensure make builds cleanly and ./rpal20 <file> works end-to-end.
- **Submission Package:** Zip as IndexNumber1_IndexNumber2.zip with Makefile at the root (no nested dirs).

4. Suggested Timeline (15-Day Plan)

Period	Phase	Key Activities	Milestone
Days 1–2	Setup	Project skeleton, Makefile, ASTNode class	Repo + build passing
Days 3–4	Lexer	Tokenizer complete, keyword list done	All token tests pass
Days 5–7	Parser	Recursive descent for full grammar	AST matches rpal.exe -ast
Days 8–10	ST Converter	All standardization rules implemented & verified	ST matches rpal.exe -st
Days 11–13	CSE Machine	All CSE rules + built-ins implemented	Sample programs produce correct output
Day 14	Testing & Fix	End-to-end diff testing on all test programs	Zero diff failures
Day 15	Report & Submit	PDF report written; zip packaged and submitted	Submission uploaded to Moodle

5. Technical Reference & Key Notes

5.1 Data Structures

- **ASTNode class** – ASTNode: String type, String value, List<ASTNode> children
- **CSE Machine** – Stack<Object>, List<Object> for control & value stacks
- **Environment** – Map<String, Object> chained with a parent reference
- **Keyword table** – Map<String, Token> for O(1) keyword lookup

5.2 Standardization Rules (Key ones)

- **let $X = E$ in P** $\rightarrow$ $\text{gamma}(\text{lambda}(X, P), E)$
- **where $P \ X = E$** $\rightarrow$ $\text{gamma}(\text{lambda}(X, P), E)$
- **fn $X . E$** $\rightarrow$ $\text{lambda}(X, E)$
- **rec $f . E$** $\rightarrow$ $\text{rec}(\text{lambda}(f, E))$
- **within** $\rightarrow$ $\text{gamma}(\text{lambda}(X2, E2), \text{gamma}(\text{lambda}(X1, E1), E0))$
- **and** $\rightarrow$ Simultaneous definitions using comma
- **@ (infix)** $\rightarrow$ $\text{gamma}(\text{gamma}(\text{op}, \text{left}), \text{right})$

5.3 CSE Machine Rules (Summary)

- Rule 1: Name lookup – push environment binding onto value stack
- Rule 2: Lambda – create closure (lambda, current-env) and push
- Rule 3: Gamma on closure – create new env, bind params, switch control to lambda body
- Rule 4: Environment exit – pop env marker, restore previous environment
- Rule 5: Conditional (beta) – evaluate bool, select true/false delta
- Rule 6: Tuple (tau) – collect n values from stack into a tuple
- Rule 7: Tuple selection – index into tuple
- Rule 8: Unary/Binary ops – pop operands, apply operator, push result
- Rule 9+: Built-in functions – Print, Order, Stern, Stem, Conc, ItoS, type-checks ...

6. Grading Checklist

[70%] Correct Implementation

- Lexer tokenizes all RPAL constructs correctly
- Parser builds AST matching `rpal.exe -ast` for all test programs
- ST conversion matches `rpal.exe -st` for all test programs
- CSE machine output matches `rpal.exe` for all test programs
- Program runs as: `./rpal20 <file>` or `java rpal20 <file>`

[10%] Code Quality & Comments

- Every function/method has a comment explaining its purpose
- Complex logic (CSE rules, standardization) is commented inline
- Consistent naming, indentation, and formatting

[20%] Report (PDF)

- Both students' names and IDs on cover page
- Program structure diagram / overview
- Function prototypes for all major components
- Design decisions and known limitations documented

7. Submission Checklist

☐	Makefile is at the ROOT of the zip (no nested directories)
☐	Executable is named <code>rpal20</code> (or main class is <code>rpal20.java</code> for Java)
☐	All source files are included with comments
☐	Report is in PDF format with both names + student IDs
☐	Zip is named: <code>IndexNumber1_IndexNumber2.zip</code>
☐	Submitted to Moodle ONLY (email submissions are ignored)
☐	make builds cleanly with no errors
☐	<code>./rpal20 rpal_test_programs/rpal_01</code> produces correct output (verify with diff)